

# Bonus Point Assignment III

## Reinforcement Learning and Learning-based Control

### Introduction

This document provides all the necessary information regarding the third bonus point assignment for the RLLBC class. This assignment is set up as a challenge where you train a car racing agent and compete against other students.

**Formalia** This is a voluntary assignment through which you can earn up to 4 bonus points for the final exam. Bonus points are only awarded if the final exam is passed. A failing grade cannot be improved using bonus points. The assignment can be completed in groups of up to 4 students. In contrast to the other bonus point assignments, you do not have to work in a group and are explicitly allowed to hand in alone. Regardless of whether you work individually or in a group, all participants must register in the same group via “*Groups Assignment 3*” on Moodle.

**Dates** Please note the following dates and deadlines, as they are strictly enforced:

1. Start of the assignment: 17<sup>th</sup> June 2026, 00:01am (one minute past midnight)
2. Submission deadline: 23<sup>rd</sup> August 2026, 11:59pm (one minute before midnight)

**Registration** To participate in this assignment, every group needs to complete the following registration procedure:

1. Register your group in RWTH Moodle.
2. Download the project folder and fill out the registration form `registration.yaml`. For that, you will need to:
  - a) Create a **private** fork from [this repository](#),
  - b) Create an access token for the repository,
  - c) Invent a team name for your group.

More detailed instructions are given in the `registration.yaml` file, please follow these carefully.

3. Submit the registration form for your group to Moodle.

**Submissions** Once you completed the registration, you can submit a solution by pushing your code to the main branch of the forked repository of your group. In regular time intervals, we will check whether you have submitted a new solution and evaluate it on a representative racetrack. Your score will be the average return that your agent earns on this track. The evaluation results will additionally be published to a leaderboard in Moodle. We plan to update the performance evaluations of your agents within a few hours after you submit a new agent. The highest score that your team achieved at any point before the deadline will count for your final score. As the evaluation will happen fully automated, there are a few things you need to pay attention to regarding your submissions:

- Ensure the structure of your repository adheres to the structure of the repository that you have originally forked. In particular, ensure that the `agent_interface.py` and `model.obj` are in their expected positions.
- Ensure that your repository does not blow up in size; it should not exceed 10 MB. **Using more than 10 MB of space without asking for permission is a reason for disqualification.**
- Provide a brief description of your agent in the `README.md` of your repository.

**Grading** You are guaranteed to receive at least two bonus point if you exceed the episodic return achieved by our agent called `DSME-DDPG-tuned` (around 62 to 64). The final grading policy will be determined after the submission deadline.

**Installation** Detailed setup instructions are provided in the `student_guide.md` file, which is located in the forked repository. Do not reuse your environment from previous assignments. Instead, set up a fresh environment as described in the guide. After successfully completing the installation, you can begin working on the task.

To *train* your model, you can extend the environment file with any packages you need. However, you have to make sure that your submitted agent can be evaluated inside the provided environment, where these additional packages will not be present during grading. Therefore, we recommend you start submitting solutions early on, such that there is plenty of time to fix any potential errors. If you believe that the evaluation environment lacks necessary packages, you can send an email to [rllbc@dsme.rwth-aachen.de](mailto:rllbc@dsme.rwth-aachen.de) to discuss a potential extension of the environment.

## Assignment

As stated before, your task is to train a competitive car racing agent. Below, we provide a guide for the race car environment, as well as an overview of the code base.

**Environment** The car racing environment was adapted from [1]. The environment consists of a race car that is placed on a track and cones on the sides of the track, which can be sensed by the car. We use a single racetrack for training and evaluation, displayed in Figure 1.

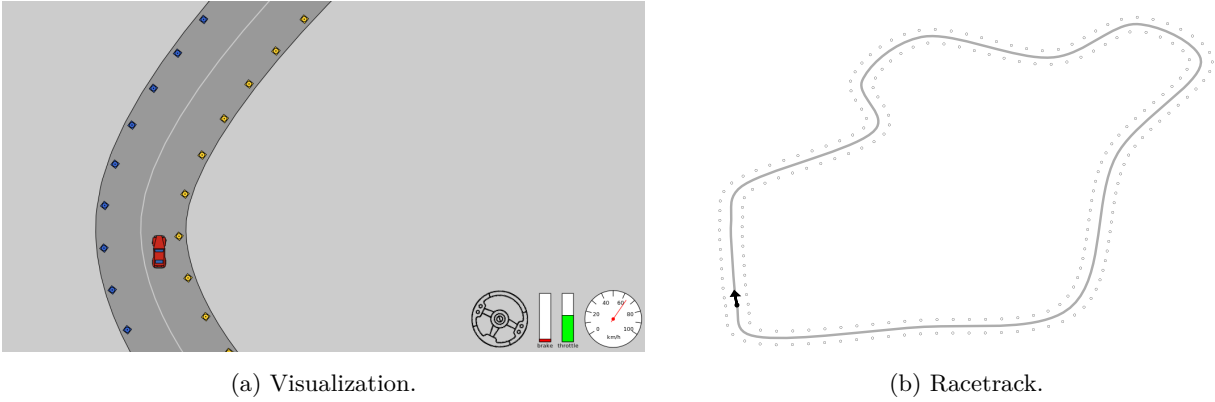


Figure 1: Overview of the racing environment.

Your goal is to make progress on the track as quickly as possible, without hitting the cones or leaving the track. This should be accomplished by using the car’s throttle, brake, and steering wheel. A reward proportional to the car’s velocity  $\mathbf{v}_{\text{ego}}$  projected onto the direction of the track  $\mathbf{n}_{\text{track}}$  is provided for each time step in which the car remains on the track. Hitting a cone does not have any effect on the car’s trajectory, but is penalized with a negative reward, proportional to the number of cones that the car hits. If the center of the car leaves the track, it leads to a reward of  $-1$  and the end of the episode. In total, the reward  $r$  for any time step amounts to:

$$r = 0.01 \cdot \mathbf{v}_{\text{ego}} \cdot \mathbf{n}_{\text{track}} - 0.2 \cdot \text{num\_collisions} - \mathbb{1}_{\text{leftTrack}} \quad (1)$$

The episode ends when 600 steps have been executed, the agent is idling (i.e. has not moved more than 0.8 meters within 5 seconds), or the car’s center leaves the track.

At each time step, our agent takes a three-dimensional action, which is shown in Table 1.

Table 1: Overview of the actions.

| Index | Action   | Range     |
|-------|----------|-----------|
| 0     | steering | $[-1, 1]$ |
| 1     | throttle | $[-1, 1]$ |
| 2     | brake    | $[-1, 1]$ |

The state consists of a description of the car  $s_{\text{car}}$  and a description of the racetrack  $s_{\text{track}}$ , such that  $s = [s_{\text{car}}, s_{\text{track}}]$ . The first 16 dimensions contain information on  $s_{\text{car}}$ , as displayed in the following Table 2:

Table 2: Overview of  $s_{\text{car}}$ .

| Index | Observation                   | Description                                              | Unit                                    | Range         |
|-------|-------------------------------|----------------------------------------------------------|-----------------------------------------|---------------|
| 0     | longitudinal velocity         | velocity in the direction the car is facing (normalized) | $\frac{\text{m/s}}{40.545 \text{ m/s}}$ | $[-1, 1]$     |
| 1     | lateral velocity              | velocity perpendicular to car's direction (normalized)   | $\frac{\text{m/s}}{40.545 \text{ m/s}}$ | $[-1, 1]$     |
| 2     | wheel angular velocity        | how quickly the wheels rotate                            | 1/s                                     | -             |
| 3     | steering angle $\beta$        | the car's steering angle (normalized)                    | $\frac{\text{rad}}{0.61 \text{ rad}}$   | $[-1, 1]$     |
| 4-5   | vehicle position $[p_x, p_y]$ | (global) position of the vehicle in x and y coordinates  | m                                       | $[0, 150]$    |
| 6     | vehicle orientation $\theta$  | car's (global) yaw angle, i.e. rotation around z-axis    | rad                                     | $[-\pi, \pi]$ |
| 7-9   | car's RGB-value               | RGB color values (in order) of the car's exterior paint  | 1                                       | $[0, 255]$    |
| 10    | traveled distance             | distance the car traveled so far in the current episode  | m                                       | $[0, 243275]$ |
| 11    | front slip                    | 1 if front wheels slip, else 0                           | 1                                       | $\{0, 1\}$    |
| 12    | rear slip                     | 1 if rear wheels are slipping, else 0                    | 1                                       | $\{0, 1\}$    |
| 13    | long. acceleration            | acceleration in the direction the car is facing          | $\text{m/s}^2$                          | -             |
| 14    | lateral acceleration          | acceleration in the direction perpendicular to the car   | $\text{m/s}^2$                          | -             |
| 15    | torque                        | rotational force generated by the engine                 | N m                                     | -             |

Note that above, "normalized" refers to dividing by the maximum possible value. Some of the variables that describe the state of the vehicle are visualized in Figure 2.

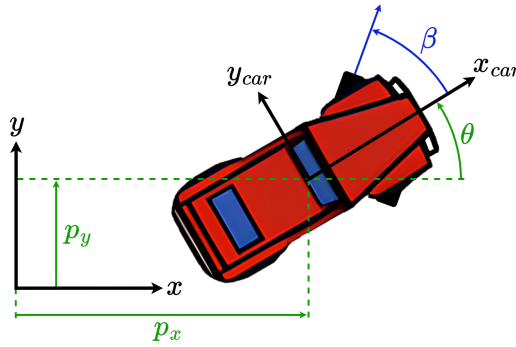


Figure 2: Vehicle in the reference coordinate system.

The description of the track via  $s_{\text{track}}$  is split in two halves; information about the cones placed along the edges of the racetrack, and information about the center line. A cone is described by 5 values, as shown in Table 3.

Table 3: Observation of the cones.

| Index | Observation    | Description                                                           |
|-------|----------------|-----------------------------------------------------------------------|
| 0     | cone spot used | 1 if cone spot is used, else 0 (happens when $< 80$ cones are sensed) |
| 1-2   | cone position  | x- and y-coord. of the cone relative to the car (normalized)          |
| 3-4   | track side     | indicate whether the cone is on the left/right track side (0 or 1)    |

In total, we can observe up to 80 cones around the car, which leads to an additional 400 dimensions in the observation space (16-20 describe cone 1, ...). We sense all cones within 75 meters to the front, 18 meters behind, and 35 meters to either side of the car. The normalization value used for the cone position is the maximum sensing range in any direction, which is 75 meters.

Additionally, we observe the center line of the track, consisting of 20 points. The points are evenly distributed over a distance of 60 meters to the front. The observation of each point is a pair of  $(x, y)$ -coordinates, defined in Table 4.

Table 4: Observation of the center line of the track.

| Index | Observation | Description                                                  |
|-------|-------------|--------------------------------------------------------------|
| 0     | x-position  | x-coord. of the center line relative to the car (normalized) |
| 1     | y-position  | y-coord. of the center line relative to the car (normalized) |

The values of the coordinates are normalized by dividing them by the maximum distance of 60 meters. Thus, in total, we add another 40 observations, leading to the 456-dimensional state space.

**Code Base** Below, you can find an overview of the provided code base.

|                      |                                                                                                                                                                                                                                                                         |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| bpa-iii/             |                                                                                                                                                                                                                                                                         |
| ├ CarEnv/            | • <b>CarEnv</b> includes the car racing environment. Please <b>do not modify</b> the environment.                                                                                                                                                                       |
| ├ models/            |                                                                                                                                                                                                                                                                         |
| ├ run_human.py       | • The <b>models</b> folder is the designated place for saving your agent. Please <b>do not move or rename</b> the folder. The file saved in this folder under the name "model.obj" is used in the evaluation. <b>Make sure that your agent is saved here correctly.</b> |
| ├ agent_interface.py |                                                                                                                                                                                                                                                                         |
| ├ training.py        |                                                                                                                                                                                                                                                                         |
| ├ try_agent.py       |                                                                                                                                                                                                                                                                         |
| └ util.py            |                                                                                                                                                                                                                                                                         |

- The file **run\_human.py** allows you to manually navigate the car around, using the arrow keys on your keyboard. This file does not hold any importance for the solution of the assignment.
- The file **agent\_interface.py** is key for the interaction of agent and environment. Your agent should be implemented here as a class called **Agent**. It must expose a method **get\_action(obs)**. This method should return an action based on the current observations provided by the environment's step function. The **agent\_interface.py** also includes the methods **convert\_obs()** and **convert\_action()**. The method **convert\_obs()** should handle all potential computations that convert the observations of the racing environment to the observations that your agent takes as input. The method **convert\_action()** provides the same functionality for the actions. The interaction of the different components is visualized in Figure 3. Please **do not put any other code in this file**, and **do not import any packages** that are not included in the provided python environment. Furthermore, **do not rename** any of the original contents of the file, as this will cause errors during evaluation.

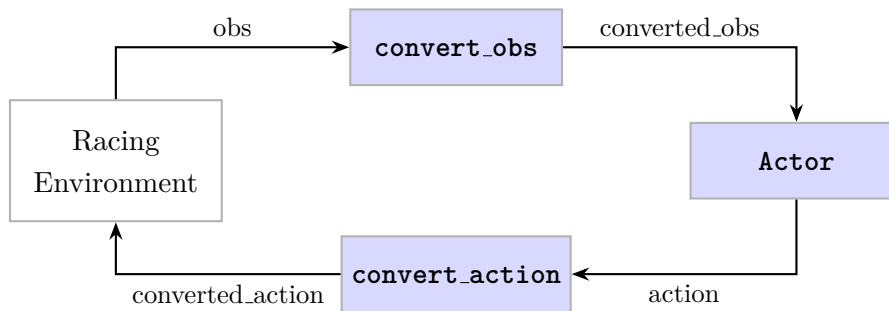


Figure 3: Interaction of the parts of your solution and the environment

- The file `training.py` should include the training algorithm of your choice. Here you can import any packages you want and there are no constraints on what you can implement. This file is not used during evaluation. However, we might consider it during grading. Therefore, we recommend keeping it up to date.
- You can use `try_agent.py` to check your agent's performance. It works similar to the evaluation used for our leaderboard. Thus, we recommend you to **regularly check whether you can load your agent** to make sure we can evaluate it. However, note that we cannot guarantee that no errors occur when we test your solution. Therefore, **try to submit your first solutions early enough** so that you have some time left to adjust your code and fix potential errors.
- The file `util.py` includes some utility functions that you may use, e.g., for saving and loading your agent. **Please do not modify** this file. We recommend taking a look at other methods in this file that might be useful.

When opening the project for the first time, you will notice that there is already some code for a DDPG agent in `agent_interface.py` and `training.py`. This code is part of a sample solution that can be a good starting point for how to structure the training of an agent and do logging. However, the performance that this solution achieves is not good. In its current state, the agent needs several million training steps to obtain a solution that is mediocre and does not surpass the baseline.

Here are a few pointers to how you might be able to train a better agent:

1. To achieve a better solution, state and action representations are key.
2. RL training performance largely depends on good hyperparameters. The hyperparameters vary with the choice of agent and training algorithm.
3. The performance also depends on the choice of the training algorithm. In Figure 4 you can find exemplary plots displaying the training performance of agents. You can notice that there are alternatives to DDPG that achieve much better results. Inspiration for alternative algorithms can be found in the [RLLBC Algorithm Library](#).

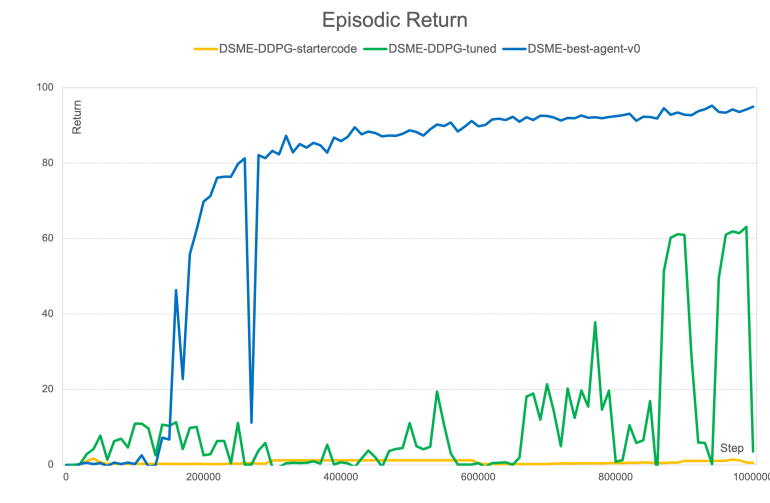


Figure 4: Performance of example agents during training.

4. Note that the performance of agents can differ between runs. Thus, it is worth training an agent multiple times and for many steps, before drawing conclusions about their performance. Of course, this requires computational resources. We recommend using the [RWTH HPC Cluster](#) to accelerate your training runs.

## References

- [1] M. Schier, C. Reinders, and B. Rosenhahn, “Learned fourier bases for deep set feature extractors in automotive reinforcement learning,” in 2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC), Sep. 2023. [Online]. Available: <https://github.com/m-schier/CarEnv>